Tienda de Deportes

Modelo de base de datos y plan de trabajo

Trabajo Final: CRUD desde código | PostgreSQL + C#

Equipo: David, Daniel y Josué

1. Qué pide el trabajo

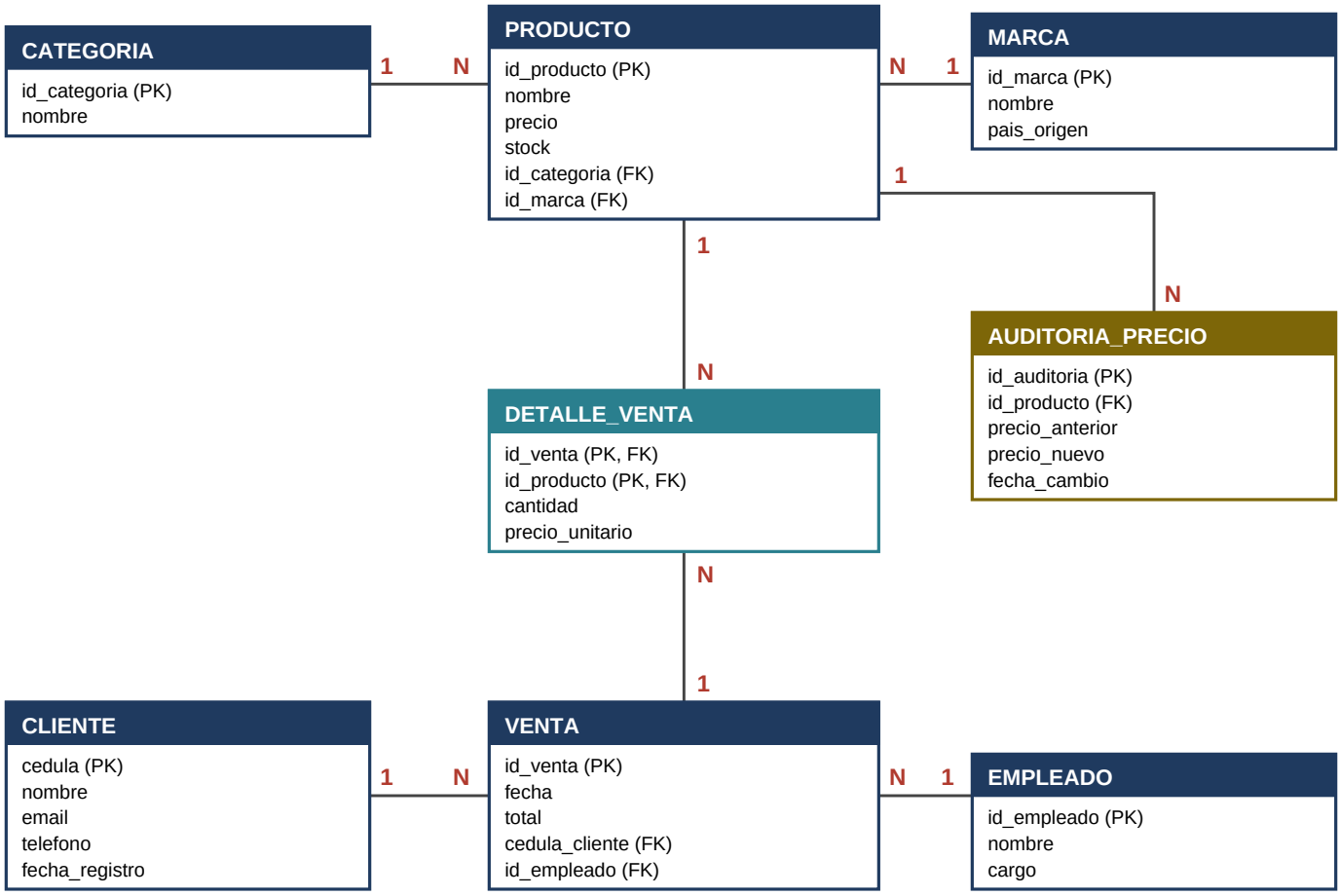
Sustentación en vivo por Teams el **10 de noviembre de 2026**, por turnos de 30 minutos acordados con el profesor. Todos los integrantes deben poder explicar todo lo realizado.

Parte	Requisito mínimo	Cómo lo cumplimos
Base de datos	Al menos 5 tablas, integridad referencial y formas normales	8 tablas, PK/FK en todas, diseño hasta 3FN
Constraints	Al menos 3 campos	CHECK, UNIQUE, NOT NULL y DEFAULT en varios campos
Índices	Al menos 2 campos	3 índices explícitos (más los de UNIQUE/PK)
Procedimientos	Al menos 2	sp_registrar_venta y sp_reponer_stock
Triggers	Al menos 2	Control de stock y auditoría de precios
Programa	CRUD a 2 tablas, sin que el programa se cancele	C# con Npgsql: CRUD de CLIENTE y PRODUCTO

2. Reglas de negocio

- Un cliente puede hacer muchas compras; cada venta es de un solo cliente.
- Un empleado registra muchas ventas; cada venta la registra un solo empleado.
- Una venta incluye varios productos y un producto puede estar en muchas ventas (relación N a M, resuelta con la tabla DETALLE_VENTA).
- Cada producto pertenece a una categoría (calzado, ropa, balones, accesorios...).
- Cada producto es de una marca (Nike, Adidas, Puma...).
- Cada cambio de precio de un producto queda registrado en una tabla de auditoría.

3. Diagrama entidad-relación



PK = llave primaria, FK = llave foránea. En las líneas, 1 = uno y N = muchos (por ejemplo, 1 CLIENTE — N VENTA). DETALLE_VENTA es la tabla intermedia que rompe la relación N a M entre VENTA y PRODUCTO.

4. Diseño de las tablas

Todas las llaves foráneas usan **ON DELETE RESTRICT**: al borrar un registro padre que tiene hijos, PostgreSQL lanza el error **23001** (restrict_violation), que es justo lo que se debe controlar en la demo. El 23503 aparece en el caso contrario: al insertar un hijo cuya FK apunta a algo que no existe.

Tabla	Columna	Tipo	Restricciones
CATEGORIA	id_categoria	INTEGER	PK, IDENTITY
	nombre	VARCHAR(50)	NOT NULL, UNIQUE
MARCA	id_marca	INTEGER	PK, IDENTITY
	nombre	VARCHAR(50)	NOT NULL, UNIQUE
	pais_origen	VARCHAR(50)	—
PRODUCTO	id_producto	INTEGER	PK, GENERATED ALWAYS AS IDENTITY
	nombre	VARCHAR(100)	NOT NULL
	precio	NUMERIC(10,2)	NOT NULL, CHECK (precio > 0)
	stock	INTEGER	NOT NULL, DEFAULT 0, CHECK (stock >= 0)
	id_categoria	INTEGER	FK a CATEGORIA, NOT NULL
	id_marca	INTEGER	FK a MARCA, NOT NULL
CLIENTE	cedula	VARCHAR(15)	PK (llave natural, la digita el usuario)
	nombre	VARCHAR(100)	NOT NULL
	email	VARCHAR(100)	UNIQUE
	telefono	VARCHAR(20)	—
	fecha_registro	DATE	DEFAULT CURRENT_DATE
EMPLEADO	id_empleado	INTEGER	PK, IDENTITY
	nombre	VARCHAR(100)	NOT NULL
	cargo	VARCHAR(50)	—
VENTA	id_venta	INTEGER	PK, IDENTITY
	fecha	TIMESTAMP	NOT NULL, DEFAULT now()
	total	NUMERIC(12,2)	DEFAULT 0 (campo derivado, ver sección 6)
	cedula_cliente	VARCHAR(15)	FK a CLIENTE, NOT NULL
	id_empleado	INTEGER	FK a EMPLEADO, NOT NULL
DETALLE_VENTA	id_venta	INTEGER	PK compuesta, FK a VENTA
	id_producto	INTEGER	PK compuesta, FK a PRODUCTO
	cantidad	INTEGER	NOT NULL, CHECK (cantidad > 0)
	precio_unitario	NUMERIC(10,2)	NOT NULL, CHECK (precio_unitario > 0)
AUDITORIA_PRECIO	id_auditoria	INTEGER	PK, IDENTITY
	id_producto	INTEGER	FK a PRODUCTO, NOT NULL

Tabla	Columna	Tipo	Restricciones
	precio_anterior	NUMERIC(10,2)	—
	precio_nuevo	NUMERIC(10,2)	—
	fecha_cambio	TIMESTAMP	DEFAULT now()

Los tipos son una propuesta; el equipo puede ajustarlos al armar el script.

5. Constraints, índices, triggers y procedimientos

Constraints

- **CHECK:** precio > 0, stock >= 0, cantidad > 0, precio_unitario > 0.
- **UNIQUE:** email de CLIENTE, nombre de CATEGORIA y de MARCA.
- **NOT NULL:** nombre en las tablas principales y las llaves foráneas.
- **DEFAULT:** stock = 0, fecha_registro = fecha actual, fecha de la venta = ahora.

Índices

En PostgreSQL los PK y UNIQUE ya crean un índice automáticamente, pero **las FK no se indexan solas**. Índices explícitos propuestos:

- producto(nombre): para búsquedas por nombre.
- producto(id_categoria): para listar productos por categoría.
- venta(cedula_cliente): para consultar las compras de un cliente.

Triggers

En PostgreSQL cada trigger necesita una **función trigger** en PL/pgSQL (que devuelve TRIGGER) y luego el CREATE TRIGGER que la llama.

Trigger	Cuándo se dispara	Qué hace
trg_control_stock	BEFORE INSERT en DETALLE_VENTA	Valida que haya stock suficiente (si no, lanza una excepción) y descuenta la cantidad vendida del stock del producto.
trg_auditoria_precio	AFTER UPDATE OF precio en PRODUCTO	Inserta en AUDITORIA_PRECIO el precio anterior, el nuevo y la fecha del cambio.

Procedimientos almacenados

Se crean con CREATE PROCEDURE y se invocan con CALL (PostgreSQL 11 o superior).

Procedimiento	Parámetros	Qué hace
sp_registrar_venta	cedula_cliente, id_empleado, id_producto, cantidad	Crea la venta y su detalle (tomando el precio actual del producto) y actualiza el total. El trigger de stock se dispara solo.
sp_reponer_stock	id_producto, cantidad	Suma la cantidad indicada al stock del producto (valida que sea mayor que 0).

6. Decisiones de diseño (para la sustentación)

precio_unitario en DETALLE_VENTA. Parece repetir el precio de PRODUCTO, pero no viola la 3FN: es el precio *al momento de la venta*. Si el producto sube de precio, las ventas anteriores no deben cambiar.

total en VENTA. Es un atributo derivado (se puede calcular sumando los detalles). Se deja como desnormalización controlada, mantenida por el procedimiento de registrar venta; la alternativa es quitarlo y calcularlo con un SELECT. El equipo debe confirmar cuál usa y saber justificarlo.

cedula como PK de CLIENTE. Como la digita el usuario, permite demostrar el error de *PK duplicada*. PRODUCTO usa *Identity*, que permite demostrar el error de *insertar valor en un campo Identity*. Así el CRUD cubre ambos casos.

Tallas. En una tienda real el stock cambia por talla (requeriría una tabla de variantes). Para esta entrega se mantiene el modelo simple, sin tallas, y se menciona como posible extensión.

7. Reparto por bloques

El trabajo se reparte por bloques, pero **los tres deben poder sustentar todo**: el profesor puede preguntarle a cualquiera sobre cualquier parte.

Bloque	Responsable	Qué incluye
A. Estructura de la BD	David	Los CREATE TABLE con PK, FK e Identity; constraints (CHECK, UNIQUE, NOT NULL, DEFAULT); índices; datos de prueba; diagrama E-R. Al terminar, apoya a Daniel con el CRUD de CLIENTE en C#.
B. Lógica de la BD	Josué	Funciones trigger y los 2 triggers (stock y auditoría de precios); los 2 procedimientos (registrar venta y reponer stock); tabla de auditoría; pruebas en SQL. Al terminar, prueba todos los casos de error en el programa y prepara el guión de la demo.
C. Programa en C#	Daniel	Proyecto C#, conexión a PostgreSQL con Npgsql, menú, CRUD de PRODUCTO, CRUD de CLIENTE (con apoyo de David) y esquema de manejo de excepciones para que el programa nunca se cancele.

Sugerencia para la sustentación

Antes de presentar, ensayar intercambiando bloques: cada uno explica la parte de un compañero. Si alguien se enreda, ahí se nota qué falta repasar.

8. Errores de PostgreSQL que el programa debe controlar

Con **Npgsql** se captura `PostgresException` y se lee su propiedad `SqlState`. Al final debe haber un `catch (Exception)` general para que el programa **nunca se cierre** por algo inesperado.

SQLSTATE	Qué pasó	Ejemplo en la tienda
23505	PK o UNIQUE duplicado	Registrar un cliente con una cédula o email que ya existe
23001	Violación de RESTRICT: borrar un padre que todavía tiene hijos (ON DELETE RESTRICT)	Borrar un cliente que ya tiene ventas; borrar una categoría que tiene productos
23503	Violación de llave foránea al insertar o actualizar: la FK apunta a algo que no existe	Crear un producto con una categoría o marca inexistente
23502	Campo obligatorio vacío (NOT NULL)	Insertar un producto sin nombre
23514	Violación de CHECK	Precio negativo o stock menor que 0
22P02	Tipo de dato inválido	Texto en un campo numérico enviado a la base de datos
428C9	Valor manual en una columna GENERATED ALWAYS	Insertar un id_producto a mano

Nota: en C# conviene validar también antes de enviar (por ejemplo con int.TryParse y decimal.TryParse) y mostrar un mensaje claro al usuario; el error 22P02 queda como segunda línea de defensa. Importante: al borrar hay que capturar 23001 (padre con hijos) y también 23503 (FK inexistente al insertar). Además, se debe leer siempre el SqlState y no el texto del mensaje, porque el texto cambia según el idioma del servidor.